二维码识别

二维码识别

- 例程实验效果简介
- 代码讲解
  - 完整代码
  - 代码结构
- find_qrcodes()
- 类 `QRCode`
  - 构造函数
  - `corners`
  - `rect`
  - `x`
  - `y`
  - `w`
  - `h`
  - `payload`
  - `version`
  - `ecc_level`
  - `mask`
  - `data_type`
  - `eci`
  - `is_numeric`
  - `is_alphanumeric`
  - `is_binary`
  - `is_kanji`

例程实验效果简介

本节我们来识别一下二维码

> **二维码和条形码的区别?**
>
> 二维码与条形码的主要差异在于编码方式和功能性：条形码是一维的、容量较小、主要用于商品识别，需专用设备扫描；而二维码是平面二维的、存储容量更大、应用范围更广、具有容错能力，且可用智能手机轻松识别，这使得二维码在现代数字生活中获得了更广泛的应用。

首先我们用二维码生成器来生成一个二维码图片（可以使用我们附件里提供的二维码生成工具，也可以在网上搜素在线生成条形码）

在输入框里输入英文或数字内容，点击生成二维码按钮即可生成



> **二维码的纠错等级**分为L、M、Q、H四个级别，纠错能力分别为7%、15%、25%和30%。当二维码部分损坏或被遮挡时，更高的纠错等级可以确保二维码仍能被正确识别和解码。但需注意，较高的纠错等级会增加二维码的密度和复杂度，使图案更密集。在实际应用中，通常根据使用环境选择合适的纠错级别

我们把生成的二维码放大



我们用CanMV IDE打开例程代码，将K230用USB连接到电脑上

点击CanMV IDE左下角的运行按钮运行例程

用摄像头对准刚刚生成的二维码，可以看到成功识别到了条形码的内容



我们在生成二维码的时候选择了高纠错级别（30%），所以我们给二维码上面覆盖一些干扰，试一下能不能正常识别



可以看到高纠错级别（30%）的二维码是有比较不错的抗干扰能力的

本节例程源码位于 【源码汇总 / 06.Codes / 02.find_qrcodes.py】

代码讲解

本节我们要用的的外设主要是摄像头模块

二维码的检测和识别 由 K230中的 find_qrcodes() 方法实现，该方法属于image模块

完整代码

```
import time
import math
import os
import gc
from media.sensor import Sensor
from media.display import Display
from media.media import MediaManager

def init_camera():
    """
    Initialize camera sensor with specified settings
    初始化摄像头传感器及其参数设置
    """
    sensor = Sensor()
    sensor.reset()  # Reset sensor to default settings | 重置传感器到默认设置
    sensor.set_framesize(width=640, height=480)  # Set resolution | 设置分辨率
    sensor.set_pixformat(Sensor.RGB565)  # Set pixel format | 设置像素格式
    return sensor

def init_display(sensor_obj):
    """
    Initialize display device
    初始化显示设备
    """
    Display.init(Display.ST7701, to_ide=True)  # Initialize ST7701 display | 初始
化ST7701显示器
    # Alternative virtual display initialization | 备用虚拟显示器初始化
    #Display.init(Display.VIRT, sensor_obj.width(), sensor_obj.height())

def process_qrcode(image, qr_result):
    """
    Process detected QR code and draw information on image
    处理检测到的二维码并在图像上绘制信息

    Args:
        image: Current frame image | 当前帧图像
        qr_result: QR code detection result | 二维码检测结果
    """
    if len(qr_result) > 0:
        # Draw rectangle around QR code | 在二维码周围画矩形
        image.draw_rectangle(qr_result[0].rect(), thickness=2)
        # Display QR code content | 显示二维码内容
        image.draw_string_advanced(0, 0, 30, qr_result[0].payload(),
                                   color=(255, 255, 255))
        print(qr_result[0].payload())

def main():
    """
    Main function to run QR code detection loop
    运行二维码检测的主函数
    """
    # Initialize system components | 初始化系统组件
    sensor = init_camera()
    init_display(sensor)
    MediaManager.init()
```

```
    sensor.run()

    # Initialize FPS clock | 初始化FPS计时器
    clock = time.clock()

    try:
        while True:
            clock.tick()  # Update FPS clock | 更新FPS计时器

            # Capture frame | 捕获图像帧
            img = sensor.snapshot()

            # Detect QR codes | 检测二维码
            qr_codes = img.find_qrcodes()

            # Process detection results | 处理检测结果
            process_qrcode(img, qr_codes)

            # Display result | 显示结果
            Display.show_image(img)
            print(clock.fps())  # Print current FPS | 打印当前FPS

    except KeyboardInterrupt:
        print("Program terminated by user")
        # Clean up resources | 清理资源
        sensor.close()
        gc.collect()

if __name__ == "__main__":
    main()
```

代码结构

本节例程主要功能和结构如下：

1. 摄像头控制：初始化摄像头，设置640x480的分辨率和RGB565像素格式
2. 图像处理：
    - 持续捕获摄像头画面
    - 在每一帧图像中寻找二维码
    - 当检测到二维码时，在画面中用矩形框标记二维码位置
    - 在画面上显示二维码内容
3. 显示输出：
    - 实时显示处理后的图像
    - 打印二维码解码内容
    - 显示当前的帧率(FPS)
4. 程序结构：
    - 采用模块化设计，将不同功能分成独立函数
    - 包含异常处理机制
    - 在程序结束时会进行资源清理

find_qrcodes()

```
image.find_qrcodes([roi])
```

该函数查找指定 ROI 内的所有二维码，并返回一个包含 `image.qrcode` 对象的列表。有关更多信息，请参考 `image.qrcode` 对象的相关文档。

为了确保该方法成功运行，图像上的二维码需尽量平展。可以通过使用 `sensor.set_windowing` 函数在镜头中心放大、使用 `image.lens_corr` 函数消除镜头的桶形畸变，或更换视野较窄的镜头，获得不受镜头畸变影响的平展二维码。部分机器视觉镜头不产生桶形失真，但其成本高于 OpenMV 提供的标准镜头，这些镜头为无畸变镜头。

- `roi` 是用于指定感兴趣区域的矩形元组 `(x, y, w, h)`。若未指定，ROI 默认为整个图像。操作范围仅限于该区域内的像素。

注意： 不支持压缩图像和 Bayer 图像。

类 `QRCode`

二维码对象由 `image.find_qrcodes` 函数返回。

构造函数

```
class image.qrcode
```

请使用 `image.find_qrcodes()` 函数创建此对象。

`corners`

```
qrcode.corners()
```

该方法返回一个包含二维码四个角的元组列表，每个元组格式为 (x, y)。四个角的顺序通常为从左上角开始，按顺时针方向排列。

`rect`

```
qrcode.rect()
```

该方法返回一个矩形元组 (x, y, w, h)，可用于其他图像处理方法，如 `image.draw_rectangle` 中的二维码边界框。

`x`

```
qrcode.x()
```

该方法返回二维码边界框的 x 坐标 (int)。您也可以通过索引 [0] 获取该值。

`y`

```
qrcode.y()
```

该方法返回二维码边界框的 y 坐标 (int)。您也可以通过索引 [1] 获取该值。

`w`

```
qrcode.w()
```

该方法返回二维码边界框的宽度 (int)。您也可以通过索引 [2] 获取该值。

`h`

```
qrcode.h()
```

该方法返回二维码边界框的高度 (int)。您也可以通过索引 [3] 获取该值。

`payload`

```
qrcode.payload()
```

该方法返回二维码的有效载荷字符串，例如 URL。您也可以通过索引 [4] 获取该值。

`version`

```
qrcode.version()
```

该方法返回二维码的版本号 (int)。您也可以通过索引 [5] 获取该值。

`ecc_level`

```
qrcode.ecc_level()
```

该方法返回二维码的错误纠正水平 (int)。您也可以通过索引 [6] 获取该值。

`mask`

```
qrcode.mask()
```

该方法返回二维码的掩码 (int)。您也可以通过索引 [7] 获取该值。

`data_type`

```
qrcode.data_type()
```

该方法返回二维码的数据类型。您也可以通过索引 [8] 获取该值。

`eci`

```
qrcode.eci()
```

该方法返回二维码的 ECI（编码指示），用于存储 QR 码中数据字节的编码。处理包含非标准 ASCII 文本的二维码时，需查看此值。您也可以通过索引 [9] 获取该值。

`is_numeric`

```
qrcode.is_numeric()
```

若二维码的数据类型为数字格式，则返回 True。

`is_alphanumeric`

```
qrcode.is_alphanumeric()
```

若二维码的数据类型为字母数字格式，则返回 True。

`is_binary`

```
qrcode.is_binary()
```

若二维码的数据类型为二进制格式，则返回 True。若需准确处理所有类型的文本，请检查 `eci` 是否为 True，以确定数据的文本编码。通常为标准 ASCII，但可能是包含两个字节字符的 UTF-8。

`is_kanji`

```
qrcode.is_kanji()
```

若二维码的数据类型为日文汉字格式，则返回 True。若返回值为 True，需自行解码字符串，因为日文汉字每个字符为 10 位，而 MicroPython 不支持解析此类文本。